CityTech TTP Summer Bootcamp

SQL Dumps and ngrok

2026-06-25

Agenda

1. Review
2. Follow-Ups
3. SQL Dumps
4. Exposing a Local Server with ngrok
5. Continue: Scattergories

Review

What are the most important things you remember from yesterday?

The only country that starts with W is Wales.

Follow Up: Good git Habits

- Always run **`git status`** before any **mutative** command (`add`, `commit`, `push`, `reset`)
- **Review** what has changed, and decide what should actually be **tracked and committed**

Avoid `git add .`

`git add .` stages **everything** at once, so you can commit things you didn't mean to.

- You might push **confidential files** (secrets, `.env`)
- Or **generated files** like `node_modules/` or `dist/`
- Safer: add files **by name**, and keep junk out with a `.gitignore`

Follow Up: Communication

Communication matters as much as code. Engineers spend much of their day **explaining, asking, and writing things down**, not just typing.

- **School group projects**: a small team, a short timeline, everyone in the room and at a similar level
- **Real-world projects**: many teams, stakeholders, and PMs; work spans weeks or months and is mostly **asynchronous** (slack, tickets, docs, code reviews, standups)
- It affects **getting hired**: interviews weigh communication as much as coding, so poor communicators often don't get the offer
- And **staying hired**: teams trust and keep clear communicators; weak ones are often the first to be let go, even if their code is fine

Follow Up: Postgres and WSL

Running into Postgres problems on **WSL**? We added a setup guide to the repo:

`reference_docs/postgres_wsl_setup_guide.md`

Exit Tickets

We'll be using **exit tickets** from now on.

In education, an **exit ticket** is a short set of tasks you finish at the **end of a session**, so the instructor can check what you learned and you can wrap up the day.

- Before you leave each day, **complete every task** on the exit ticket
- If you're **absent**, complete them **before** you come in next time. Don't just review the slides; complete the action items.

Intro to SQL Dumps

A **SQL dump** is a file of SQL statements that can **recreate a database**: its schema, its data, or both.

Why use one:

- **Backup**: save a snapshot you can restore if something breaks
- **Migrate / share**: move a database to another machine or teammate (like the Scattergories project)

Clean vs Non-Clean Dumps

- A **full / clean** dump includes `DROP` statements, so restoring **wipes the existing tables first** and replaces them with an exact copy
- A **non-clean** dump only **creates and inserts**, so restoring onto tables that already exist will **error or duplicate**

Dumps in pgAdmin 4

Create a dump (Backup):

- Right-click the database, choose **Backup...**
- Pick a filename and format (**Plain** gives a `.sql` script)
- For a full/clean dump, enable **"Clean before restore"** in the options
- Click **Backup**

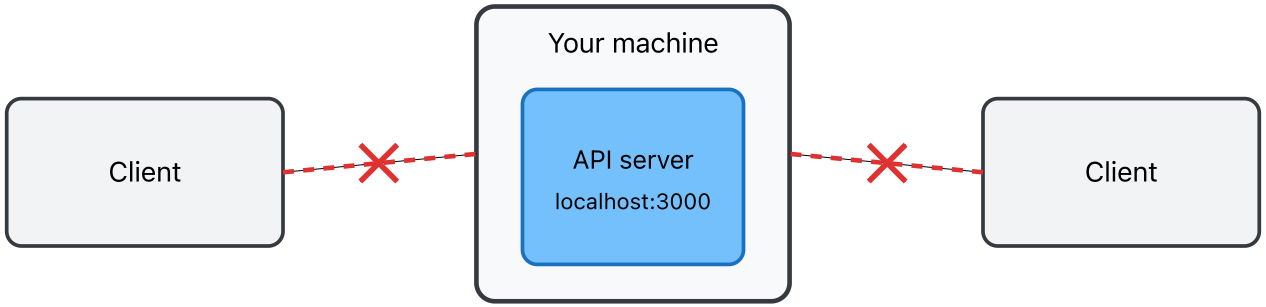
Restore a dump:

- Right-click the target database, choose **Restore...**
- Select your dump file and format, then click **Restore**

Official guide: https://www.pgadmin.org/docs/pgadmin4/latest/backup_and_restore.html

Local Network Access

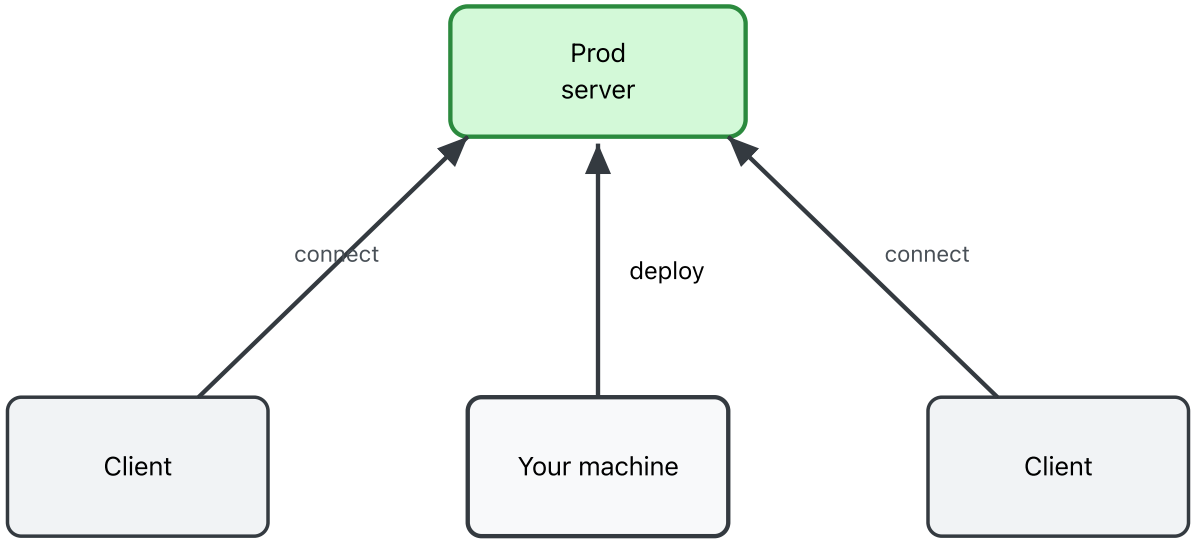
A server on `localhost` is only reachable **from your own machine**; other machines (and the internet) cannot connect to it.



So a **client on another machine** cannot reach your API directly.

Production (Prod) Level Solution

Run the server on an **always-on machine** in the cloud. You **deploy** your code to it, and **clients connect to the server**, not to your laptop.



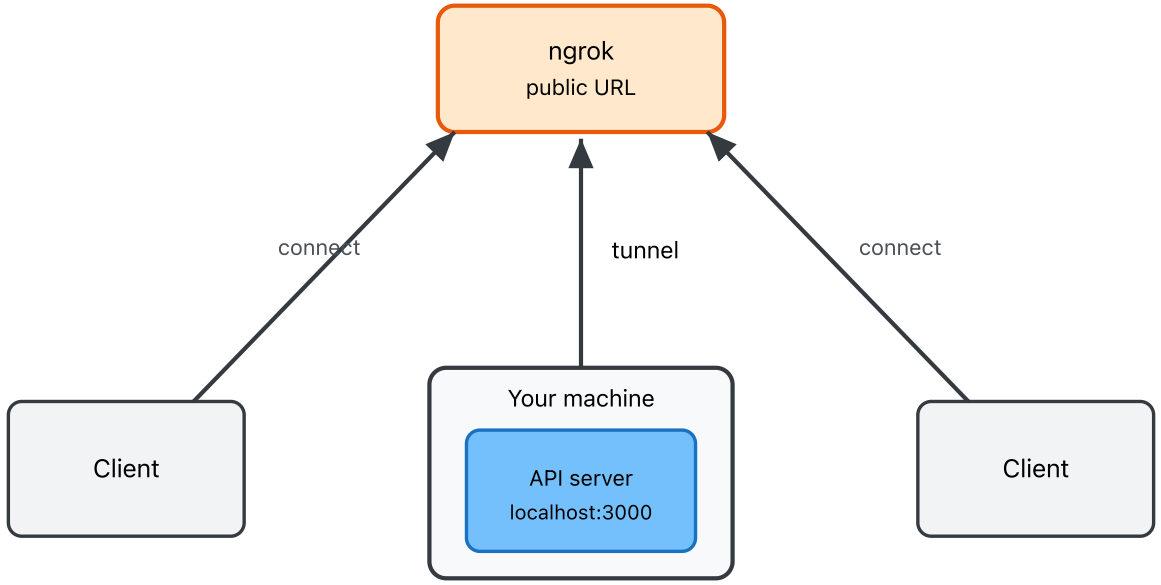
What Prod Needs

A real deployment is not free or automatic. It needs:

- **Server provisioning and maintenance**: set up the machine, then handle updates, security, monitoring, and fixing it when it breaks
- **Money to keep it running**: the server costs money **around the clock**, even when **no one is using it**

An Alternative Solution: ngrok

`ngrok` opens a **tunnel** from a **public URL** to your local server. Clients hit the public URL, and ngrok forwards their requests down to your API on `localhost`.



Benefits: fast to set up (one command, no server to provision) and free on the free tier.

What Are Tunnels?

A **tunnel** is a connection that lets outside traffic reach a machine that normally can't be reached directly.

- Like a **P.O. box**: people send mail to a public address, and it's forwarded to your private machine
- Your machine reaches out to open the tunnel, so requests can find their way back to it
- **SSH** can create tunnels too; **ngrok** is a ready-made one: a public URL with a single command

Demo: ngrok in Action

I'll run a **Claude-generated Scattergories** server, expose it with **ngrok**, and we'll play together.

- I share the public **ngrok URL**
- You send a room code, username, and answers to it
- Everyone's answers land in one shared game

Example: `demos/scattergories-example`

Setting Up ngrok

Set it up on your own machine:

1. **Create a free account** at ngrok.com
2. **Install the CLI** in the same place your server runs: `brew install ngrok` (macOS) or the **Linux** install via `apt` (inside **WSL** on Windows)
3. **Add your authtoken**: `ngrok config add-authtoken <YOUR_TOKEN>`
4. **Smoke test**: serve something locally, then tunnel it: `python3 -m http.server 8000`, then `ngrok http 8000`, and open the public URL

Official guide: https://ngrok.com/docs/getting-started

Continue: Scattergories

Keep building your **Scattergories** group project.

- Get the core working first: new game, submit answer, and the validation rules
- Then take on the **tiered learning goals**
- Full spec: `project_specs/scattergories.md`

Today's Exit Ticket

- [ ] Set up a **Postgres database** with tables
- [ ] **Export** the database as a dump
- [ ] **Import** that dump
- [ ] Create an **ngrok account**
- [ ] Use the **ngrok CLI** to expose a local port
- [ ] Run a **smoke test** on ngrok
- [ ] Make progress on the **coding project**
- [ ] Submit your **project repo url** via the Google Form in slack